

claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf\_b531eaeef-39c\agent-a7c56af28b84e3efc.jsonl`
- SHA-256: `38f2aa4e35e81e3728f7ad426ab330585b851da5ae96bb210a187af91c128550`
- Source modified: `2026-06-19T17:02:13+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_b531eaeef-39c`
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`

Transcript

1. user (2026-06-19T16:59:09.583Z)

You are an adversarial design reviewer. Be skeptical; the goal is to find real flaws BEFORE we build.

CONTEXT ? badminton/racket-sports rally detector. Pipeline: WASB shuttle DETECTOR (heatmap ? per-frame candidate list) ? single-target online TRACKER (collapses to ONE (x,y) per frame) ? velocity windower ? rallies.

MEASURED (GX010142, owner's anchor clip, on the 21-25s window, from the cached det_raw.jsonl detector candidates):

- WASB's detector emits ≥ 2 candidates on 31% of detected frames (cached, never used past the tracker).
- During the 21-25s rally: 93% of frames carry the ADJACENT-court shuttle (x-norm ~ 0.88 , what the tracker followed),
 - 83% ALSO carry a PROMINENT-court shuttle (x-norm ~ 0.56), 76% carry BOTH at once.
- A greedy nearest-neighbour reconstruction of the prominent-court candidates yields a CONTINUOUS rally arc
 - (75% window coverage, 431px vertical excursion, 8.6px median per-frame jump, ~ 2.4 s sustained)

? VISUALLY CONFIRMED

as a real foreground-court rally (player mid-stroke ? net ? second player), which the single-target tracker discarded.

PROPOSED DESIGN ("the Which problem"):

Split the tracker's two conflated jobs:

- Layer 1 "see everything": a MULTI-TARGET linker over the ALREADY-CACHED det_raw.jsonl candidates ? a SET of shuttle
 - tracks per region (top-K), each with trajectory + duration + motion-energy. Pure-Python over the cache, ZERO GPU,
 - WASB stays frozen (MIT).
- Layer 2 "pick the match's story": score each track by court-membership (the existing prominent-court polygon +
 - point_in_polygon) + rally-likeness (motion energy, vertical excursion, ≥ 2 net-crossings/Gate-A, ends on shuttle-floor-contact) and SELECT the track(s) whose story is the match's; feed those to the windower.

Claimed benefits: recall-POSITIVE (recovers the discarded prominent rally ? a G1 miss) AND precision-positive (drops the adjacent rally) ? unlike today's court_gate which is a subtractive post-filter on the single track (precision-only). Also claimed to SUBSUME the "ghost" (a stuck-pixel false-lock track has ~ 0 motion energy ? never selected). Lands default-OFF, flag-gated, measured on cached candidates + golden labels with no GPU; single-track tracker stays the default.

YOUR LENS:

Attack the NOISE problem. 31% of frames have ≥ 2 detector candidates but many are NOISE (reflections, lines, shirts), not second shuttles. A multi-target linker over noisy candidates can spawn many spurious tracks. Will a HEURISTIC selection objective (motion energy + court + net-crossings) reliably separate shuttle-tracks from noise-tracks, or does this need a learned model? Quantify the tuning/overfitting risk. Mitigations.

Return concrete, technically-grounded risks with severity + mitigations. Mark kills_design=true ONLY for a fundamental unmitigable flaw.

2. assistant (2026-06-19T16:59:12.034Z)

I'll investigate the actual code and cached data to ground this review in reality before assessing the NOISE-problem risks.

3. user (2026-06-19T16:59:15.725Z)

Your configuration specifies to merge with the ref 'refs/heads/docs/court-floor-rally-end' from the remote, but no such ref was fetched.

4. user (2026-06-19T16:59:15.998Z)

ABLATION_LAYER.md
ANALYZERS_AND_RECONCILERS.md
BACKLOG.md
CODE_AUDIT_AND_TEST_HARDENING.md
CODE_MAP.md
COMMERCIALIZATION.md
DEFERRED_HARDENING_PLAN.md
DESKTOP_APP_PLAN.md
EVALUATION.md
EXECUTION_POLICY.md
EXPERIMENT_HARNESS.md
FIELD_VISIT_ANSWER_SHEET.md
FIELD_VISIT_PLAYBOOK.md
GOLDEN_DATA_SHARING.md
GOLDEN_REGRESSION_FIXTURES.md
GOLDEN_SET_IMPLEMENTATION.md
GOLDEN_SET_PHASE1_VIDEOS.md
GOLDEN_SET_VENDORING.md
GOLDEN_VIDEOS.md
HARDENING_LOOP_HANDOFF.md
HOSTING_PLAN.md
HOW_RALLY_DETECTION_WORKS.md
I18N_PLAN.md
KHELSUTRA_PRODUCT_OVERVIEW.md
MULTI_SIGNAL_FUSION_PLAN.md
NEXT_STEPS.md
OWNED_MODEL_IMPLEMENTATION_PLAN.md
PERSONALIZATION_PLAN.md
PER_VIDEO_OUTPUT_LAYOUT.md
PLATFORM_ARCHITECTURE.md
PMF_FIELD_SIGNALS.md
PMF_MARKET_RESEARCH.md
POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
PRODUCT_FIELD_ASSOCIATE_FLYER.md
PROJECT_DOC_TEMPLATE.md
PROMINENT_COURT_DETECTION.md
QUALITY_ITERATIONS.md
R1_MILESTONE_LAUNCH_REPORT.md
R2_EVAL_METRIC_DESIGN.md
RALLY_DETECTION_GAP_CLOSURE.md
RALLY_DETECTION_QUALITY_REPORT.md
RALLY_QUALITY_RESEARCH.md

README.md
REAL_FOOTAGE_VALIDATION_REPORT.md
ROADMAP.md
ROORA_LABELING_GUIDELINES.md
STORAGE_SHARING_MODEL.md
TRACKNET_WSL_SETUP.md
UI_ALPHA_EXECUTION_PLAN.md
UI_PROPOSAL.md
---RALLY GATE---
backend/pipeline/detectors/rally_gate.py

5. assistant (2026-06-19T16:59:19.751Z)

The pull failed (remote branch ref mismatch) ? I'll proceed read-only on local state and note it. Let me examine the actual code that this design depends on: the detector candidate cache, the existing tracker, the court polygon gate, and Gate-A.

6. user (2026-06-19T16:59:20.636Z)

```
1      """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3      WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4      the opponent for service after losing a point, that single trajectory looks like a
5      rally. The discriminator is structural, not duration-based: a real rally has the
6      shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7      single service feed crosses ~once.
8
9      This module is camera-agnostic and needs NO new model ? it works on the trajectory
10     we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11     axis with the larger shuttle spread) and the net position (median shuttle coord on
12     that axis ? the shuttle clusters at the two ends and crosses the net region often,
13     so the median sits near the net), then counts sign changes of (coord - net) across
14     the visible points inside a candidate window, with a deadband to ignore jitter.
15
16     Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17     Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18     windows from trajectory_to_action_windows, or to fold into it later.
19     """
20     from __future__ import annotations
21
22     from dataclasses import dataclass
23     from typing import List, Optional, Sequence, Tuple
24
25     Interval = Tuple[float, float]
26
27
28     @dataclass
29     class GatedWindow:
30         start: float
31         end: float
32         crossings: int
33         visible_points: int
34         kept: bool
35
36
37     def _spread(vals: Sequence[float]) -> float:
38         """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39         if len(vals) < 2:
40             return 0.0
41         s = sorted(vals)
42         lo = s[max(0, int(0.05 * (len(s) - 1)))]
43         hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44         return hi - lo
45
46
```



```

110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

7. user (2026-06-19T16:59:21.379Z)

```

1      # Prominent-Court Detection ? kill background-court false positives (multicourt)
2
3      > **Status:** `DRAFT` ? design + experiment **owner-gated for any default flip** .
**Owner:** Avi . **Created:** 2026-06-19 . **Last updated:** 2026-06-19
4      > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** §8 progress tracker is the source of truth; pointer in memory
`current-state`.
6      > **Relation to existing docs:** attacks gap **G2** ("precision ? local over-fires,
worst on multicourt") from [`RALLY_DETECTION_GAP_CLOSURE.md`](RALLY_DETECTION_GAP_CLOSURE.md);
it is the **shuttle-side** counterpart of the person-side **Gate B / court mask** in
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md). Complements the held-shuttle
**`shuttle_player_colocation`** (G3) ? court-localization (G2) and held-shuttle (G3) are
independent precision levers.
7      > **Honesty note:** claims are tagged `[verified]` (checked vs code/measured) /
`[design]` (proposed). Every signal lands **flag-gated, default-OFF**, promoted only on measured
lift per [`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md) + [[decision-rigor-norm]].
8
9      ---
10
11     ## 0. TL;DR
12     In **multicourt** footage the camera sees several courts. The shuttle detector (WASB)
tracks *any* shuttle in frame, so a rally on a **background / adjacent court** becomes a
predicted rally on the **main match** ? a false positive. The fix: identify the **prominent
court** (the foreground court that occupies the majority of the frame, where the match being
filmed is played) and **keep only rallies whose shuttle lives inside it**. Mechanically this re-
defines the shuttle signal from `(x, y, visible)` to `(x, y, visible-AND-inside-the-prominent-
court)` *before* windowing ? so a background-court shuttle reads as "not there" and never forms
a rally. A flat 2D floor polygon is the MVP; a **pseudo-3D play-volume** (court floor + the air
column above it) is the refinement so a high clear over the prominent court isn't wrongly
rejected. Lands as a **default-OFF experiment** ; the flip is gated on a Launch Report ([[launch-
report-convention]]).
13
14     ## 1. Problem & motivating example `[verified]`
15     - **Measured gap (G2):** local over-fires ~2x vs Gemini, and the over-fire is **worst on
multicourt** ? even among long (>7s) rallies, multicourt over-fires **1.58x** (background-court
games are *real* long rallies, just on the wrong court). See `RALLY_DETECTION_GAP_CLOSURE.md` §3
+ the P7 long-rally lens.
16     - **Concrete anchor ? GX010142 rally #1 (2026-06-19).** Owner observation: "the first
rally is just a shuttle flying in the non-prominent court? while the prominent-court players
have not started." Confirmed from the cached WASB trajectory
(`output/wasb/GX010142_1080p__wasb.csv`): rally #1 (`0.72?3.65s`, 2.94s) has a shuttle
centroid at **meanX ? 1616** on a 1920-wide frame (?84% to the right) ? spatially apart from the
central cluster where most rallies live (meanX ? 1000?1300). Several other suspicious rallies
(#3, #9, #12, #16) cluster in the same far-right band ? consistent with a **right-side adjacent
court**.
17     - **Why a 1D threshold is not enough `[verified]`:** the courts do not separate cleanly
on X or Y alone (rally Y-centroids overlap 250?620 px across courts; the far-right band mixes
with foreground play). The courts are **2D regions** under perspective ? so we need the court
*geometry*, not a coordinate cutoff. This is the owner's "pseudo-3D / court model" intuition,
validated by the data.
18
19     ## 2. Goal & non-goals

```

```

20     - **Goal:** on multicourt clips, drop rallies whose shuttle is outside the prominent
court, **without losing real prominent-court rallies** ? measured per-video, no regression on
single-court clips (where it must be a no-op or a do-no-harm).
21     - **Non-goals (v1):** multi-camera; tracking players across courts; full court-line
homography for scoring; anything biometric. Single-court footage is explicitly a **no-op** (one
court ? everything is "prominent").
22
23     ## 3. Decisions locked
24     | # | Decision | Rationale |
25     |---|-----|-----|
26     | D1 | The prominent court is a **2D region (polygon) in image space**, refined to a
**pseudo-3D play volume** | 1D thresholds don't separate courts under perspective ($1) |
27     | D2 | Gate the **shuttle trajectory** by the region **before windowing** (re-define
visibility) | Stops a background rally at the source ? no window ever forms there. Reuses
`point_in_polygon` |
28     | D3 | **Default-OFF, flag-gated; single-court = no-op** | Zero-regression rollout
([[weak-signals-retained]]); no behaviour change unless a polygon is supplied + the flag set |
29     | D4 | Reuse the existing **`court_polygon`** config + `point_in_polygon`; do NOT invent
a new geometry stack | `FusionConfig.court_polygon` + `fusion_features.point_in_polygon` already
exist `verified` |
30     | D5 | The prominent court can be **supplied (config) OR auto-derived**; auto-derivation
is its own measured step | Config polygon is the cheapest first experiment; auto is the product
path |
31
32     ## 4. Design
33
34     ### 4.1 The signal change (the core experiment)
35     Today the windower consumes the raw WASB trajectory: a per-frame `(Frame, Visibility, X,
Y)`. The change is a **pre-windowing filter**:
36
37     ```
38     for each frame f: if shuttle_visible(f) and NOT inside_prominent_court(X_f, Y_f): mark
f as NOT visible
39     ```
40
41     A background-court shuttle is then indistinguishable from "no shuttle" ? the velocity
windower never opens a rally there. This is purely **subtractive** on the shuttle stream (it can
only remove visibility, never add it), so on a correctly-drawn prominent court it cannot
manufacture rallies ? only suppress out-of-court ones. (Mirror of `rally_gate`'s "do-no-harm"
discipline.)
42
43     ### 4.2 Identifying the prominent court (four routes, cheapest first)
44     1. **Configured polygon** (MVP, `[design]?buildable now`): a normalized `[[x,y],?]`
court polygon per camera/clip, exactly the existing `FusionConfig.court_polygon`. Draw it once
from the dense shuttle/player cluster. Good enough to validate the whole concept on GX010142 +
the multicourt golden clips.
45     2. **Player-location auto-derivation** (`[design]`): run
`PersonDetector.detections_per_frame` `verified` exists, take the **largest, most-central,
most-persistent** player cluster (the foreground players are bigger boxes lower in frame), and
fit the prominent-court polygon to where **those** players' feet live. "Prominent = where the
match's players are." This is why the owner asked to **extract person features** ? players
anchor the court.
46     3. **Shuttle-density auto-derivation** (`[design]`): the prominent court is the densest
sustained shuttle-activity region; cluster the visible trajectory and take the dominant mode.
Person-free fallback.
47     4. **Court-line detection** (`[design]`, heaviest): Hough/segmentation court lines ? the
largest court quad. Most accurate, most work; deferred.
48
49     v1 ships route 1 (config) + measures; route 2 (player-anchored) is the product upgrade.
50
51     ### 4.3 Pseudo-3D play volume (the refinement)
52     A flat floor polygon rejects a shuttle at the top of a high clear **over the prominent
court** (it projects above the floor quad). The pseudo-3D model treats the prominent court as a
**floor quad + a vertical air column**: accept a shuttle if its image point lies within the
floor polygon **expanded upward** by a height envelope (the max plausible shuttle height

```

projected through the camera). Practically: the floor quad plus an upward `pad` (perspective-aware ? larger near the net/far baseline). Start with a generous fixed upward pad (cheap, MVP); upgrade to a per-camera height envelope only if measurement shows real rallies being clipped. This keeps the floor tight (rejects adjacent-court floor play) while not clipping legitimate high shuttles.

```
53
54     ### 4.4 Reuse map `[verified]`
55     | Need | Existing code |
56     |---|---|
57     | point-in-region test |
`backend/pipeline/detectors/fusion_features.py::point_in_polygon` |
58     | normalized court polygon config |
`backend/config/models.py::FusionConfig.court_polygon` (today: person/Gate-B only) |
59     | player boxes per frame |
`backend/pipeline/detectors/person_detector.py::PersonDetector.detections_per_frame` |
60     | trajectory parse + windowing |
`backend/pipeline/detectors/tracknet_runner.parse_trajectory_csv`, `backend/eval/windowing.py` |
61     | per-video scoring | `backend/eval/rally_seg_eval.py` (+ the `--stratify` multicourt
split, #220) |
62
63     New code is small: a `prominent_court_gate(points, polygon, height_pad)` that filters
trajectory points, threaded into the windowing input behind a default-OFF flag.
64
65     ### 4.5 Extension ? deterministic rally-END via shuttle-floor contact (owner note,
2026-06-19) `[design]`
66     The same court geometry that localizes the prominent court also unlocks a
**deterministic rally-END signal** ? the cheapest, most certain way to know a point is over:
**the shuttle hit the floor.** This attacks **G3** (rally-END over-extension), where today's
shuttle-velocity windowing runs the window long because "shuttle still moving ? rally over."
67
68     - **Introduce two geometric primitives:**
69     - **The white court lines.** A badminton court is bounded by **white, straight lines
forming a rectangle** (with the perspective trapezoid in image space). Detect them ? Hough lines
/ line-segment detection filtered to white, long, court-consistent segments ? to recover the
**court quad** (which also *is* the prominent-court floor polygon of §4.2 route 4; one detector
serves both).
70     - **The floor plane.** The court lines define the **floor** (the plane the players
stand on). The shuttle in flight lives *above* this plane; a shuttle *at* the plane (within the
trapezoid, or just outside it) is **on the floor**.
71     - **The rally-END rule (deterministic):** a rally ends when the shuttle makes **floor
contact** ? its trajectory descends to and **stops/rests at the floor level** (a near-zero-
velocity terminus at the court-line plane), OR a shuttle that goes to the floor **outside the
prominent court** = an out-of-bounds landing (also point-over). Unlike velocity heuristics, a
shuttle resting on the floor is **unambiguous** ? the point is over.
72     - **Why it's strong + cheap:** no new model, no person stack ? it reuses the WASB
trajectory (which already gives the shuttle's descending `(x, y)` + the `Visibility` drop when
it settles) plus the court-line geometry. It is the **shuttle-side** rally-END signal,
complementary to the **player-state** rally-END (player kinematics / held-shuttle) in
`RALLY_DETECTION_GAP_CLOSURE.md` §4 Lever A ? fuse both for robustness (a shuttle can settle
off-camera; a player can pick up early).
73     - **Subtlety to measure:** distinguish a true floor landing (point over) from a shuttle
that **dips low but is played back up** (a tight net shot / low retrieve). The court-line plane
+ a brief **dwell** at floor level (the shuttle rests, doesn't re-accelerate up) is the
discriminator; tune the dwell window on golden clips.
74
75     This is tracked as **C8** below and cross-links gap-closure **G3**. It shares the court-
line detector with the prominent-court polygon (§4.2 route 4), so the two land together once
court-line detection exists.
76
77     ## 5. Experiment plan (measurement)
78     Land the gate flag-gated, then measure on the **cached trajectories** (CPU, no GPU/re-
detect):
79     1. **GX010142 sanity:** draw a prominent-court polygon; confirm **rally #1 disappears**
and the central rallies survive. The litmus the whole idea is born from.
80     2. **Multicourt golden** (`mahadevpura_2`, GX010128, Badminton_BXH_2, Boxhill_Doubles`):
```

? rally-count, and once those clips are golden-labelled, ? precision / ? recall vs golden via `rally\_seg\_eval --stratify` (single vs multicourt). The win condition: multicourt precision up, **recall flat** (we drop FPs, not real rallies).

81 3. **Single-court no-op:** on single-court clips with no polygon (or a full-frame polygon) the gate is a verified no-op ? assert zero change.

82 4. **Dual-eval discipline** ([[eval-dual-set-regression]]): no per-video regression on either eval set.

83

84 Promotion to default-ON is a **separate, owner-gated** decision with a Launch Report (both rulers + the over-seg guard), never bundled with the experiment PR.

85

86 ## 6. Risks & limits

87 - **polygon is camera-dependent.** A hand-drawn polygon doesn't generalize across setups ? route 2/3 (auto-derivation) is the real product path; v1 config-polygon is for *validation*.

88 - **Shuttle height** (\$4.3): too-tight a floor clips high clears; the pseudo-3D pad mitigates, measurement tunes it.

89 - **The prominent court can change** (camera re-aim mid-clip): out of scope v1 (assume static per clip).

90 - **Not a recall fix.** This only removes out-of-court FPs; it does nothing for missed in-court rallies (G1) ? and must be proven not to *cost* recall.

91

92 ## 7. Open questions (owner / data)

93 1. Auto-derivation anchor: **player-location** (route 2) vs **shuttle-density** (route 3) as the default once validated? *(decides: data, post-experiment)*

94 2. Height-envelope: fixed upward pad vs per-camera perspective model ? is the fixed pad enough? *(decides: measurement)*

95 3. Do we expose the prominent court as a **one-time per-camera setup** (draw-once UI) or always auto? *(decides: owner/product)*

96

97 ## 8. Deliverables & progress tracker ? source of truth

98 Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, off `origin/master`, no stacking.

99

ID	Deliverable	Depends on	Gated?	Status	PR
C0	This design doc	? No	? this PR		
C1	`prominent_court_gate(points, polygon, height_pad)` + unit tests (pure, point-in-polygon)	? No (default-OFF)	? ?		
C2	Thread the gate into the windowing input behind a default-OFF flag (`court_polygon` + `gate_shuttle_to_court`)	C1	No (default-OFF)	? ?	
C3	GX010142 litmus: rally #1 dropped, central rallies kept (measured)	C2	No	? ?	
C4	Extract person features (`detections_per_frame`) on GX010142 ? player-cluster = prominent court; measure `shuttle_player_colocation` too	? No	? ?		
C5	Multicourt-golden measurement (? count now; ? precision/recall vs golden once labelled)	C2, labels	? owner labels	? ?	
C6	Auto-derive the polygon (route 2 player-anchored)	C4	? owner	? ?	
C7	Launch decision (default-ON) with a Launch Report	C3, C5	? owner	? ?	
C8	**Court-line / floor detection** ? (a) auto court polygon, (b) **deterministic shuttle-floor-contact rally-END** (\$4.5, targets G3)	? ? owner	? ?		

111

112 ## 9. Definition of done

113 - [] The gate exists, is **default-OFF**, unit-tested, and a verified no-op on single-court / no-polygon.

114 - [] GX010142 rally #1 (and the far-right adjacent-court band) is dropped while the central rallies survive `[measured]`.

115 - [] On the multicourt golden clips, multicourt **precision improves with recall flat** vs golden labels (no per-video regression).

116 - [] A winning configuration is flipped default-ON only via a Launch Report ? flip `Status ? DONE`, archive.

117

118 ## 10. References

119 **Code [verified]:** `fusion\_features.py` (`point\_in\_polygon`, `person\_in\_court`), `person\_detector.py` (`detections\_per\_frame`), `config/models.py` (`FusionConfig.court\_polygon`), `eval/windowing.py`, `eval/rally\_seg\_eval.py` (`--stratify`).


```

120      **Docs:** [ `RALLY_DETECTION_GAP_CLOSURE.md` ](RALLY_DETECTION_GAP_CLOSURE.md) (G2),
[ `MULTI_SIGNAL_FUSION_PLAN.md` ](MULTI_SIGNAL_FUSION_PLAN.md) (Gate B / court mask),
[ `EXPERIMENT_HARNESS.md` ](EXPERIMENT_HARNESS.md),
[ `QUALITY_ITERATIONS.md` ](QUALITY_ITERATIONS.md).
121
122      ---
123
124      ### Changelog
125      - 2026-06-19 ? Created from the owner's GX010142 rally-#1 (background-court FP)
observation + the measured multicourt over-fire (G2). Validated the 2D/pseudo-3D need from the
cached trajectory; mapped the reuse onto the existing `court_polygon` + `point_in_polygon`
machinery.
126      - 2026-06-19 ? Added $4.5 (owner note): a **deterministic rally-END signal via shuttle-
floor contact** ? detect the white court lines ? the floor plane ? a shuttle resting on the
floor (or landing out-of-court) ends the rally (targets G3). Shares the court-line detector with
the prominent-court polygon; tracked as C8.
127

```

8. assistant (2026-06-19T16:59:25.248Z)

Now let me look at the actual detector candidate cache format and the tracker that collapses to a single point, plus the windowing. This is critical for assessing whether the noise problem is real.

9. user (2026-06-19T16:59:28.912Z)

```

././claude/worktrees/boxhill17/output/wasb/Boxhill_Doubles_proxy__9a35ffdfdbde_wasb_wasbcache/det
_raw.jsonl
././claude/worktrees/boxhill17/output/wasb/GX010132_proxy__430d77131262_wasb_wasbcache/det_raw.js
onl
././claude/worktrees/boxhill17/output/wasb/GX010135_proxy__f10da909f0a4_wasb_wasbcache/det_raw.js
onl
././claude/worktrees/boxhill17/output/wasb/GX010137_1080p__74eedbc35d73_wasb_wasbcache/det_raw.js
onl
././claude/worktrees/boxhill17/output/wasb/GX010138_1080p__1b24bb67717f_wasb_wasbcache/det_raw.js
onl
././claude/worktrees/boxhill17/output/wasb/GX010139_1080p__4229be621384_wasb_wasbcache/det_raw.js
onl
././claude/worktrees/boxhill17/output/wasb/GX010140_1080p__80cfa1d20990_wasb_wasbcache/det_raw.js
onl
././claude/worktrees/boxhill17/output/wasb/GX010141_1080p__daaf8af19864_wasb_wasbcache/det_raw.js
onl
././claude/worktrees/boxhill17/output/wasb/GX020094_proxy__08471270f285_wasb_wasbcache/det_raw.js
onl
././claude/worktrees/boxhill17/output/wasb/GX020140_1080p__d8230f6d57b1_wasb_wasbcache/det_raw.js
onl
---
././claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_infer.py
././claude/worktrees/bhi-execpolicy/backend/pipeline/detectors/wasb_infer.py
././claude/worktrees/boxhill17/backend/pipeline/detectors/wasb_infer.py
././claude/worktrees/gap-doc/backend/pipeline/detectors/wasb_infer.py
././claude/worktrees/hardchop/backend/pipeline/detectors/wasb_infer.py
././claude/worktrees/pr-stitch/backend/pipeline/detectors/wasb_infer.py
././claude/worktrees/recdoc/backend/pipeline/detectors/wasb_infer.py
././claude/worktrees/resweep/backend/pipeline/detectors/wasb_infer.py
././claude/worktrees/tier1-windowing/backend/pipeline/detectors/wasb_infer.py
././claude/worktrees/w2-safeguard/backend/pipeline/detectors/wasb_infer.py
---TRACKER---
backend/config/models.py
backend/pipeline/detectors/court_gate.py
backend/pipeline/detectors/person_detector.py
backend/pipeline/detectors/wasb_infer.py
backend/pipeline/segmenters/yolo_hybrid.py

```

10. user (2026-06-19T16:59:28.920Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12     - per_frame is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13     index (so it aligns directly with the shuttle trajectory's frame ? no clip re-
timing).
14     - A shuttle point has frame, visible, x, y (pixels) ? the WASB
trajectory shape.
15     - court_polygon is normalised 0-1 [(x, y), ...] or None (None ? no mask,
every
16     detection counts ? the player-count-only mode).
17     - net is (axis 'x'|'y', position 0-1 normalised) or None (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except mean_player_motion (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
```

```

55         return (0.0, 0.0)
56     x1, _y1, x2, y2 = box
57     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
72         (0,0)."""
73         if court_polygon is None:
74             return True
75         if frame_width <= 0 or frame_height <= 0:
76             return False
77         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                       frame_height: float,
81                       court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82         """Per-frame count of in-court persons (one entry per sampled frame)."""
83         return [
84             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85                 court_polygon))
86             for _frame_idx, dets in sorted(per_frame.items())
87         ]
88
89     def _median(vals: Sequence[float]) -> float:
90         if not vals:
91             return 0.0
92         s = sorted(vals)
93         n = len(s)
94         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
104         flicker)."""
105         if not counts:
106             return 0.0
107         return sum(1 for c in counts if c >= min_players) / len(counts)
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                          frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112         normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113         not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:

```

```

115         return 0.0
116     frames = sorted(per_frame.keys())
117     steps: List[float] = []
118     for a, b in zip(frames, frames[1:]):
119         prev = {tid: box for tid, box in per_frame[a]}
120         for tid, box in per_frame[b]:
121             if tid in prev:
122                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125     if not steps:
126         return 0.0
127     return sum(steps) / len(steps)
128
129
130 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                     frame_height: float, net: Optional[Net],
132                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137     0.0.
138     """
139     if net is None or not per_frame:
140         return 0.0
141     axis, pos = net
142     both = 0
143     for _frame_idx, dets in per_frame.items():
144         near, far = False, False
145         for _tid, box in dets:
146             if not person_in_court(box, frame_width, frame_height, court_polygon):
147                 continue
148             fx, fy = _foot_norm(box, frame_width, frame_height)
149             coord = fx if axis == "x" else fy
150             if coord < pos:
151                 near = True
152             else:
153                 far = True
154         if near and far:
155             both += 1
156     return both / len(per_frame)
157
158 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                    pad_frac: float) -> bool:
160     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161     width/height (the 'adjacent / in-hand' tolerance)?"""
162     x1, y1, x2, y2 = box
163     px = pad_frac * frame_width
164     py = pad_frac * frame_height
165     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                               frame_width: float, frame_height: float,
170                               pad_frac: float = 0.03) -> float:
171     """Fraction of (visible shuttle frames that were also person-sampled) in which the
172     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
173     ~0**
174     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
175     """
176     if not per_frame or frame_width <= 0 or frame_height <= 0:

```

```

176         return 0.0
177     aligned = 0
178     colocated = 0
179     for p in shuttle_points or []:
180         if not getattr(p, "visible", False):
181             continue
182         dets = per_frame.get(int(p.frame))
183         if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
184             continue
185         aligned += 1
186         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187             for _tid, box in dets):
188             colocated += 1
189     return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
201 shuttle_points,
202                                frame_width: float, frame_height: float,
203                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
204 None,
205                                net: Optional[Net] = None,
206                                min_players: int = 2) -> Dict[str, float]:
207     """Aggregate the per-window person features from per-frame detections + the in-
208 window
209 shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
210     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
211     return {
212         "players_in_court": players_in_court(counts),
213         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
214 court_polygon),
215         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
216         "in_court_ratio": in_court_ratio(counts, min_players),
217         "shuttle_player_colocation": shuttle_player_colocation(
218             per_frame, shuttle_points, frame_width, frame_height),
219     }

```

11. assistant (2026-06-19T16:59:32.202Z)

Let me examine the actual det_raw.jsonl format and the WASB tracker logic, plus look at the GX010142 cache to understand the candidate structure and the actual noise characteristics.

12. user (2026-06-19T16:59:33.562Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8     Frame, Visibility, X, Y
9
10    It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11    imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12    (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.

```

```

13
14 Resumable across reboots
15 -----
16 The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17 the slow, expensive part; the CPU tracker pass that turns detections into a
18 trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19 resume mid-stream). We exploit that split:
20
21 * The detector pass writes its per-window raw outputs incrementally to
22 ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23 records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24 most ``--flush-every`` windows of GPU work instead of the whole pass.
25 * On restart we replay the cached windows, skip the DataLoader ahead to the first
26 un-cached window, and continue. Once every window is cached, the detector pass
27 is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28 the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29 not the GPU hour.
30
31 The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32 to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33 weights, sport, window size, or frame count change.
34
35 Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
37     ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39     every 1000] [--fresh]
40
41 """
42 import os
43 import os.path as osp
44 import csv
45 import glob
46 import json
47 import argparse
48 import logging
49 from collections import defaultdict
50 from contextlib import contextmanager
51
52 logger = logging.getLogger(__name__)
53
54 CACHE_VERSION = 1
55 _DET_FILE = "det_raw.jsonl"
56 _MANIFEST_FILE = "manifest.json"
57
58 def _build_cfg(weights: str, sport: str):
59     """Compose the WASB eval config via Hydra, overriding model/weights/device."""
60     from hydra import compose, initialize
61     with initialize(config_path="configs", version_base=None):
62         cfg = compose(config_name="eval", overrides=[
63             f"dataset={sport}",
64             "model=wasb",
65             f"detector.model_path={weights}",
66             "runner.device=cuda",
67             "runner.gpus=[0]", # this box has a single GPU; default config assumes
68             ])
69     return cfg
70
71 def _frame_id(path: str) -> int:
72     """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
73     180)."""
74     stem = osp.splitext(osp.basename(path))[0]
75     digits = "".join(ch for ch in stem if ch.isdigit())

```

```

74         return int(digits) if digits else -1
75
76
77     # ----- #
78     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
79     # ----- #
80     def _cache_paths(cache_dir: str):
81         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
82
83
84     def default_cache_dir(out_csv: str) -> str:
85         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
86         d = osp.dirname(osp.abspath(out_csv))
87         stem = osp.splitext(osp.basename(out_csv))[0]
88         return osp.join(d, stem + "_wasbcache")
89
90
91     def _atomic_write_text(path: str, text: str) -> None:
92         """Write then rename so a crash mid-write can't leave a half-written file."""
93         tmp = path + ".tmp"
94         with open(tmp, "w") as f:
95             f.write(text)
96             f.flush()
97             os.fsync(f.fileno())
98         os.replace(tmp, path)
99
100
101     def _det_plain(det) -> list:
102         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
103         list."""
104         xy = det["xy"]
105         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
106
107     def _dets_to_tracker(plain_list):
108         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
109         array,
110         the tracker does vector math / np.linalg.norm on it)."""
111         import numpy as np
112         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
113                 for p in plain_list]
114
115     def write_manifest(cache_dir: str, manifest: dict) -> None:
116         _, mpath = _cache_paths(cache_dir)
117         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
118
119
120     def read_manifest(cache_dir: str):
121         _, mpath = _cache_paths(cache_dir)
122         if not osp.exists(mpath):
123             return None
124         try:
125             with open(mpath) as f:
126                 return json.load(f)
127         except (json.JSONDecodeError, OSError):
128             return None
129
130
131     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
132                             frames_in: int, frames_total: int) -> bool:
133         """A cache is reusable only if the run that produced it matches this run's
134         inputs."""
135         if not manifest or manifest.get("version") != CACHE_VERSION:
136             return False

```

```

136     return (
137         manifest.get("frames_dir") == osp.abspath(frames_dir)
138         and manifest.get("weights") == weights
139         and manifest.get("sport") == sport
140         and manifest.get("frames_in") == frames_in
141         and manifest.get("frames_total") == frames_total
142     )
143
144
145 def reset_cache(cache_dir: str) -> None:
146     """Drop any existing cache so the next run starts clean."""
147     det_jsonl, mpath = _cache_paths(cache_dir)
148     for p in (det_jsonl, mpath):
149         if osp.exists(p):
150             os.remove(p)
151
152
153 def load_and_clean_cache(cache_dir: str):
154     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
155     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
156     prefix so a trailing half-written line from a crash can't corrupt future appends.
157
158     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
159     [x, y, score, scale] candidates (accumulated across every window the frame is in,
160     in window order ? identical to a non-resumed run).
161     """
162     det_jsonl, _ = _cache_paths(cache_dir)
163     det_by_fid = defaultdict(list)
164     valid: list = []
165     if osp.exists(det_jsonl):
166         with open(det_jsonl, "r") as f:
167             for line in f:
168                 s = line.strip()
169                 if not s:
170                     continue
171                 try:
172                     obj = json.loads(s)
173                 except json.JSONDecodeError:
174                     break # truncated trailing line (crash mid-
flush)
175                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
176                     break
177                 valid.append(s)
178                 for fid, plain in obj["f"]:
179                     det_by_fid[int(fid)].extend([list(p) for p in plain])
180                 # Guarantee a clean append boundary by rewriting only the good prefix.
181                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
182     return len(valid), det_by_fid
183
184
185 def _append_windows(fh, objs) -> None:
186     """Append window records as JSONL and durably flush (bounded loss on crash)."""
187     for o in objs:
188         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
189     fh.flush()
190     os.fsync(fh.fileno())
191
192
193 def _atomic_write_trajectory(out_csv: str, rows) -> None:
194     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
195     tmp = out_csv + ".tmp"
196     with open(tmp, "w", newline="") as f:
197         w = csv.writer(f)
198         w.writerow(["Frame", "Visibility", "X", "Y"])
199         w.writerows(rows)

```



```

200         f.flush()
201         os.fsync(f.fileno())
202         os.replace(tmp, out_csv)
203
204
205         # ----- #
206         # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
207         # ----- #
208         _STREAM_FRAME_FMT = "{:08d}.png"
209
210
211     def synthetic_frame_path(index: int) -> str:
212         """Stable, index-encoding frame name used by the streaming path.
213
214         It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
215         downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
216         frames came from disk or from the in-memory stream. The streaming image loader
217         inverts this name back to an index to fetch the decoded frame.
218         """
219         return _STREAM_FRAME_FMT.format(int(index))
220
221
222     def build_windows(frames: list, frames_in: int):
223         """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
224         + checkpoint). Pure list math, identical for the disk and streaming paths.
225
226         Returns a list of windows, each a list of ``frames_in`` consecutive paths
227         (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
228         caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
229         ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
230         """
231         return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
232
233
234     class SequentialFrameStore:
235         """In-memory sliding buffer over a forward-only frame reader, sized for the
236         detector's sliding-window access pattern (peak O(window) frames, not O(video)).
237
238         The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
239         ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
240         in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
241         start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
242         boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
243         far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
244         earliest index any not-yet-finished window can still ask for) and decode each source
245         frame exactly once via the forward-only ``reader(index)``.
246         """
247
248         def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
249             self._reader = reader
250             self._total = int(total)
251             self._window = max(1, int(window))
252             self._buf: dict = {}
253
254             # On a resumed run the detector's first needed frame is `start_index`; begin
255             # pulling there so the reader can skip (seek past) the already-cached prefix
256             # instead of decoding 0..start_index-1 just to throw them away.
257             self._next = int(start_index) # next index not yet pulled from the reader
258             self._max_seen = int(start_index) - 1
259             self._floor = int(start_index) # lowest index we still keep (never evict
260             below)
261
262         def get(self, index: int):
263             index = int(index)
264             if index < self._floor:
265                 raise KeyError(

```

```

264         f"frame {index} already evicted (below floor {self._floor}); the access
265         "
266         f"pattern must stay within a window of the highest frame seen")
267     if index >= self._total:
268         raise KeyError(f"frame {index} out of range (total {self._total})")
269     # Pull forward from the reader until the requested index is buffered.
270     while self._next <= index:
271         self._buf[self._next] = self._reader(self._next)
272         self._next += 1
273     if index > self._max_seen:
274         self._max_seen = index
275     # Evict frames older than the earliest a still-running window could re-request:
276     # once we've seen index `m`, no future window starts before `m - (window-1)`.
277     new_floor = max(self._floor, self._max_seen - (self._window - 1))
278     for k in range(self._floor, new_floor):
279         self._buf.pop(k, None)
280     self._floor = new_floor
281     return self._buf[index]
282
283 def _index_from_synthetic_path(path: str) -> int:
284     """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
285
286     Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
287     in lockstep with the on-disk naming scheme.
288     """
289     return _frame_id(path)
290
291 def _bgr_to_pil_rgb(frame_bgr):
292     """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
293     EXACT
294     PIL RGB image WASB's disk path produces.
295
296     The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
297     Image.open(PNG).convert('RGB')``;
298     because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
299     COLOR_BGR2RGB)``
300     bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
301     streamed
302     frame indistinguishable from the on-disk one to everything downstream.
303     """
304     import cv2
305     from PIL import Image
306     return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
307
308 def _make_streamed_read_image(store, real_read_image):
309     """Build the ``read_image`` replacement used during a streaming detector pass.
310
311     A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
312     image,
313     matching ``read_image``'s real return type); any path that doesn't parse to an index
314     falls
315     through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
316     testable
317     without the WSL-only ``dataloaders`` package.
318     """
319     def _read_image(path, *args, **kwargs):
320         idx = _index_from_synthetic_path(path)
321         if idx < 0:
322             return real_read_image(path, *args, **kwargs)
323         return _bgr_to_pil_rgb(store.get(idx))
324     return _read_image

```

```

321
322     @contextmanager
323     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
int = 0):
324         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
325         frames during the detector pass, byte-for-byte as it would over real PNGs.
326
327         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
328         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
329         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
330         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
331         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
332         ``osp.exists`` guard is sidestepped for synthetic stream paths.
333
334         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
335         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
336         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
337         reference).
338
339         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
340         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
341         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
342         original reader is restored on exit.
343         """
344         from dataloaders import dataset_loader as _dl
345         store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
346         real_read_image = _dl.read_image
347         # Rebind read_image in the consuming module for the duration of the detector pass;
348         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
349         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
350         try:
351             yield store
352         finally:
353             _dl.read_image = real_read_image # type: ignore[assignment]
354
355
356     # ----- #
357     # Inference
358     # ----- #
359     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
360            limit: int = 0, batch_size: int = 8, num_workers: int = 4,
361            log_every_batches: int = 50, cache_dir: "str | None" = None,
362            flush_every: int = 1000, fresh: bool = False,
363            frame_loader=None, source_key: "str | None" = None) -> str:
364         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
365
366         Two equivalent ways to supply pixels (output is bit-identical between them):
367
368         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
369         globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
370         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)

```

```

371     frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
372     pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
373     reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
374     ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
375     to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
376     ``cvtColor(bgr, BGR2RGB)`` , verified byte-identical). Streaming forces
``num_workers=0``
377     (the in-process shim + buffer can't cross worker processes).
378
379     ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
380     frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
381     after reboot still matches.
382     """
383     import time
384     import torch
385     from torch.utils.data import DataLoader
386     from detectors import build_detector
387     from trackers import build_tracker
388     from dataloaders import build_img_transforms, build_seq_transforms
389     from dataloaders.dataset_loader import ImageDataset
390     from utils import Center
391
392     streaming = frame_loader is not None
393
394     cfg = _build_cfg(weights, sport)
395     frames_in = int(cfg["model"]["frames_in"])
396     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
397     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
398
399     if streaming:
400         # `frames` is already the ordered list of (synthetic) frame paths.
401         if not isinstance(frames, (list, tuple)):
402             raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
403         frames = list(frames)
404         frames_dir = source_key or "stream"
405     else:
406         frames_dir = frames
407         frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
glob.glob(osp.join(frames_dir, "*.jpg")))
408         if limit:
409             frames = frames[:limit]
410         if len(frames) < frames_in:
411             where = frames_dir if not streaming else "video stream"
412             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
413
414         # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
415         samples = []
416         for window in build_windows(frames, frames_in):
417             annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
418             samples.append({"frames": window, "annos": annos})
419         windows_total = len(samples)
420
421         # ---- cache setup / resume decision ----- #
422         if cache_dir is None:
423             cache_dir = default_cache_dir(out_csv)
424         os.makedirs(cache_dir, exist_ok=True)
425         det_jsonl, _ = _cache_paths(cache_dir)
426         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
427         manifest = None if fresh else read_manifest(cache_dir)
428         resume = manifest_compatible(

```

```

429         manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
430         frames_in=frames_in, frames_total=len(frames),
431     )
432     if resume:
433         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
434         logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
435     else:
436         if manifest is not None:
437             logger.info("cache incompatible with this run -> starting fresh")
438             reset_cache(cache_dir)
439             windows_done, det_by_fid = 0, defaultdict(list)
440
441     base_manifest = {
442         "version": CACHE_VERSION,
443         "frames_dir": cache_key,
444         "weights": weights,
445         "sport": sport,
446         "frames_in": frames_in,
447         "frames_total": len(frames),
448         "windows_total": windows_total,
449         "windows_done": windows_done,
450     }
451     write_manifest(cache_dir, base_manifest)
452
453     # ---- detector pass over the un-cached windows ----- #
454     remaining = samples[windows_done:]
455     if remaining:
456         _, transform_test = build_img_transforms(cfg)
457         try:
458             _, seq_transform_test = build_seq_transforms(cfg)
459         except Exception:
460             seq_transform_test = None
461         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
462                         transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
463         # Streaming forces single-process loading: the in-memory frame store + the
464         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
465         loader_workers = 0 if streaming else num_workers
466         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
467         detector = build_detector(cfg)
468
469         from contextlib import ExitStack
470
471         t0 = time.time()
472         done = windows_done
473         win_idx = windows_done
474         log_every = max(1, log_every_batches)
475         flush_n = max(1, flush_every)
476         pending = []
477         logger.info(f"detecting on {len(remaining)} remaining windows "
478                 f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
479                 f"source={'video-stream' if streaming else 'frames-dir'})...")
480         det_f = open(det_jsonl, "a")
481         try:
482             with ExitStack() as stack:
483                 stack.enter_context(torch.no_grad())
484                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
485                 # in-memory decoded frame for that synthetic path; every other line of
486                 # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.

```

```

487         # The detector's first needed frame is `windows_done` (window i starts
at
488         # frame i), so prime the store there for a resume.
489         if streaming:
490             stack.enter_context(
491                 _streaming_read_image_patch(frame_loader, len(frames),
492                                             window=frames_in,
start_index=windows_done))
493         for bi, batch in enumerate(loader):
494             imgs, _hms, trans = batch[0], batch[1], batch[2]
495             img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
496             batch_results, _ = detector.run_tensor(imgs, trans)
497             nb = imgs.shape[0]
498             for ib in range(nb):
499                 frames_payload = []
500                 for ie in sorted(batch_results[ib].keys()):
501                     p = img_paths[ie][ib]
502                     fid = _frame_id(p)
503                     plain = [_det_plain(d) for d in batch_results[ib][ie]]
504                     frames_payload.append([fid, plain])
505                     det_by_fid[fid].extend(plain)
506                 pending.append({"w": win_idx, "f": frames_payload})
507                 win_idx += 1
508             done += nb
509             if len(pending) >= flush_n or done >= windows_total:
510                 _append_windows(det_f, pending)
511                 pending = []
512                 base_manifest["windows_done"] = done
513                 write_manifest(cache_dir, base_manifest)
514             if (bi + 1) % log_every == 0 or done >= windows_total:
515                 el = time.time() - t0
516                 new_done = done - windows_done
517                 rate = new_done / el if el > 0 else 0.0
518                 eta = (windows_total - done) / rate if rate > 0 else 0.0
519                 logger.info(f"{done}/{windows_total} "
520                             f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
521                             f"elapsed {el:.0f}s | ETA {eta:.0f}s")
522             if pending:
523                 _append_windows(det_f, pending)
524                 base_manifest["windows_done"] = done
525                 write_manifest(cache_dir, base_manifest)
526         finally:
527             det_f.close()
528             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
529     else:
530         logger.info("detector cache already complete -> tracker-only re-run")
531
532     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
533     tracker = build_tracker(cfg)
534     tracker.refresh()
535     rows = []
536     for fid in sorted(det_by_fid.keys()):
537         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
538         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
539
540     _atomic_write_trajectory(out_csv, rows)
541     visible = sum(1 for _, v, *_ in rows if v)
542     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
543     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
544     return out_csv
545
546
547 def extract_frames(video_path: str, frames_dir: str) -> str:

```

```

548     """Decode a video to PNG frames named by source frame index (00000000.png ...).
549
550     Idempotent: if the dir already holds the expected number of frames (matching the
551     container's reported frame count) we skip re-decoding ? re-extraction after a
552     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
553     extracting with ffmpeg on the host and passing --frames_dir.
554     """
555     import cv2
556     os.makedirs(frames_dir, exist_ok=True)
557     cap = cv2.VideoCapture(video_path)
558     if not cap.isOpened():
559         raise SystemExit(f"cannot open video: {video_path}")
560     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
561     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
562     if expected > 0 and existing >= expected:
563         cap.release()
564         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
565         return frames_dir
566     i = 0
567     while True:
568         ok, frame = cap.read()
569         if not ok:
570             break
571         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
572         i += 1
573     cap.release()
574     logger.info(f"extracted {i} frames -> {frames_dir}")
575     return frames_dir
576
577
578     def _probe_frame_count(video_path: str) -> int:
579         """Container-reported frame count via cv2 (0 if unknown). Used to size the
580         stream."""
581         import cv2
582         cap = cv2.VideoCapture(video_path)
583         if not cap.isOpened():
584             raise SystemExit(f"cannot open video: {video_path}")
585         n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
586         cap.release()
587         return n
588
589     def make_video_frame_loader(video_path: str):
590         """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
591
592         ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
593         long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
594         detector's sliding-window order); it reads sequentially and only uses a frame-seek
595         to
596         skip forward when a *resume* starts past the current position. Each ``cap.read()``
597         returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
598         would
599         yield (PNG is lossless), so the detector sees identical pixels to the disk path.
600
601         Returns the count hint from the container so the caller can build the frame list;
602         ``release`` closes the capture.
603         """
604         import cv2
605         cap = cv2.VideoCapture(video_path)
606         if not cap.isOpened():
607             raise SystemExit(f"cannot open video: {video_path}")
608         count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
609         state = {"pos": 0} # next index cap.read() will return
610
611         def frame_loader(index: int):

```

```

610         index = int(index)
611         if index < state["pos"]:
612             raise RuntimeError(
613                 f"streaming decode is forward-only; got index {index} after
{state['pos']}")
614         if index > state["pos"]:
615             # Forward skip only happens when resuming past cached windows. Seek once.
616             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
617             state["pos"] = index
618             ok, frame = cap.read()
619             if not ok:
620                 raise RuntimeError(f"failed to decode frame {index} from {video_path}")
621             state["pos"] += 1
622             return frame
623
624     def release():
625         cap.release()
626
627     return frame_loader, count_hint, release
628
629
630     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
631                             limit: int = 0, batch_size: int = 8,
632                             log_every_batches: int = 50, cache_dir: "str | None" = None,
633                             flush_every: int = 1000, fresh: bool = False) -> str:
634         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
635         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
636         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
637         ``run``'s docstring for why.
638         """
639         frame_loader, count_hint, release = make_video_frame_loader(video_path)
640         try:
641             if count_hint <= 0:
642                 raise SystemExit(
643                     f"could not determine frame count for {video_path}; cannot stream "
644                     f"(fall back to frame extraction with --frames_dir)")
645             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
646             source_key = "video:" + osp.abspath(video_path)
647             return run(frame_list, weights, out_csv, sport, limit,
648                       batch_size=batch_size, num_workers=0,
649                       log_every_batches=log_every_batches, cache_dir=cache_dir,
650                       flush_every=flush_every, fresh=fresh,
651                       frame_loader=frame_loader, source_key=source_key)
652         finally:
653             release()
654
655
656     def main():
657         ap = argparse.ArgumentParser()
658         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
659         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
660         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
661         ap.add_argument("--stream-video", action="store_true",
662                         help="FAST PATH: decode --video on the fly and feed frames straight
to "
663                             "the detector (no PNG extraction). Output-equivalent to the
disk path.")
664         ap.add_argument("--weights", required=True)
665         ap.add_argument("--out", required=True)
666         ap.add_argument("--sport", default="badminton")
667         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")

```



```

668     ap.add_argument("--batch-size", type=int, default=8)
669     ap.add_argument("--num-workers", type=int, default=4)
670     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671     ap.add_argument("--cache-dir", default=None,
672                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
673     ap.add_argument("--flush-every", type=int, default=1000,
674                     help="fsync the detector cache every N windows (bounds reboot
loss)")
675     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676     args = ap.parse_args()
677
678     if args.stream_video:
679         if not args.video:
680             raise SystemExit("--stream-video requires --video")
681         run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
682                             batch_size=args.batch_size,
683                             log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
684                             flush_every=args.flush_every, fresh=args.fresh)
685         return
686
687     frames_dir = args.frames_dir
688     if args.video:
689         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
690     if not frames_dir:
691         raise SystemExit("provide --frames_dir or --video")
692     run(frames_dir, args.weights, args.out, args.sport, args.limit,
693         batch_size=args.batch_size, num_workers=args.num_workers,
694         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
695         flush_every=args.flush_every, fresh=args.fresh)
696
697
698     if __name__ == "__main__":
699         main()
700

```

13. assistant (2026-06-19T16:59:37.473Z)

This is very informative. The detector emits candidate dicts with `{xy, score, scale}`. The score is the heatmap peak confidence ? crucial for the noise question. Let me inspect the actual cached candidate data to ground the noise characterization, and look at the WASB tracker's online logic to understand what it does with multiple candidates.

14. user (2026-06-19T16:59:40.652Z)

```

num caches found: 11
output\wasb\GX010142_1080p__c4cfa94a792f_wasb_wasbcache\det_raw.jsonl
output\wasb\Adarsh v Avi__bb961d0d0cff_wasb_wasbcache\det_raw.jsonl
output\wasb\GX010125_1080p__89605e80ed01_wasb_wasbcache\det_raw.jsonl

```

15. assistant (2026-06-19T16:59:43.727Z)

The GX010142 cache exists. Let me analyze the actual candidate data to ground the noise question with real numbers ? score distributions of primary vs secondary candidates, how many candidates per frame, scale distribution.

16. user (2026-06-19T16:59:54.413Z)

```

windows: 29935 frames touched: 29937
frames with >=1 det: 11750 multi(>=2): 9617 pct multi: 81.8%

```

```
cand-count hist (top): {1: 2133, 2: 2537, 3: 5951, 4: 541, 5: 209, 6: 320, 7: 43, 8: 12}
PRIMARY (top-score per frame) n=11750 min=0.500 p25=12.062 med=14.430 p75=15.420 max=20.168
mean=12.866
SECONDARY (2nd-score)      n=9617 min=0.500 p25=11.437 med=13.666 p75=14.775 max=17.255
mean=12.273
ALL candidate scores      n=30622 min=0.500 p25=11.160 med=13.678 p75=14.877 max=20.168
mean=12.137
```

17. assistant (2026-06-19T16:59:59.130Z)

This is a critical, design-relevant finding. The secondary candidates are NOT low-confidence noise ? their score distribution (median 13.67) is barely below the primary (median 14.43). The detector's own confidence does NOT separate the "real second shuttle" from "noise" candidates. Let me dig deeper: look at the score gap between primary and secondary on the same frame, and the scale field, and check the WASB tracker logic to understand what threshold it uses.

18. user (2026-06-19T17:00:08.220Z)

```
total candidates: 30622  scale hist: {0: 30622}
candidates with score<=1.0 (near floor): 826 (2.7%)
candidates with score>1.0: 29796 (97.3%)
primary-secondary score GAP: n=9617 med=1.003 p25=0.416 p75=2.147  (small gap => detector cannot
rank them)
multi-cand frames where 2nd score>=8.0 (a confident 2nd peak): 8376 (87.1% of multi)
```

19. assistant (2026-06-19T17:00:12.890Z)

This sharpens the noise question dramatically. 87% of multi-candidate frames have a confident second peak (score >= 8.0), and the median primary-secondary gap is only ~1.0 on a scale where typical scores are 12-15. The detector's confidence is essentially uninformative for ranking shuttle-vs-noise. Let me now look at the WASB online tracker to see what gate it applies, and check the windowing + how the design proposes to score tracks.

20. user (2026-06-19T17:00:18.679Z)

```
---WSL?---
---local copy of tracker?---
./backend/pipeline/detectors/wasb_infer.py
```

21. user (2026-06-19T17:00:18.695Z)

```
1      """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3      **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4      ?
5      `fusion_golden` ? the golden feature JSONs the ablation/compare path re-scores) was
6      built
7      on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8      deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9      pipeline
10     (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11     merge_gap=2.0,
12     min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
13     is
14     measured on candidates **production never serves**, so a "win" need not transfer. This
15     is
16     exactly how Gate-A scored **0.074** on the proposal set yet **0.008** on the served
17     set
18     (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
19
20     **The fix.** One module owns the two named windowings as :class:`WindowingPreset`, and
21     the
22     SERVED preset is **single-sourced from the same Pydantic config defaults production
23     reads**
```

```

15     (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16     build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17     *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18     operating point a given deployment truly serves.
19
20     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21     windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22     ``per_user_gate``). This module is the what windowing; that one is the promotion
gate.
23
24     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25     caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26     """
27     from __future__ import annotations
28
29     from dataclasses import dataclass
30     from typing import Any, Dict, List, Mapping, Tuple
31
32     from backend.config.models import IndexingConfig
33
34     Interval = Tuple[float, float]
35
36     __all__ = [
37         "WindowingPreset",
38         "SERVED",
39         "PROPOSAL",
40         "PRESETS",
41         "served_windowing_from_config",
42         "build_windows",
43         "windows_to_intervals",
44     ]
45
46
47     @dataclass(frozen=True)
48     class WindowingPreset:
49         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52         "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54         knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset is the
windowing.
55         """
56
57         name: str
58         velocity_thresh: float
59         merge_gap: float
60         min_window_duration: float
61         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63         max_window_duration: float = 0.0
64         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.

```

```

66         hard_cap: bool = False
67         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
68         #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69         inactivity_end: float = 0.0
70         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
71         inactivity_rally_thresh: float = 0.08
72         inactivity_min_density: float = 0.34
73         #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74         #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75         #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76         #: factor keeps it surgical (a normal long rally is left alone).
77         blob_fallback: bool = False
78         blob_factor: float = 4.0
79
80     def as_tuple(self) -> Tuple[float, float, float]:
81         """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82         ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83         bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84         return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86     def to_dict(self) -> Dict[str, Any]:
87         return {
88             "name": self.name,
89             "velocity_thresh": self.velocity_thresh,
90             "merge_gap": self.merge_gap,
91             "min_window_duration": self.min_window_duration,
92             "max_window_duration": self.max_window_duration,
93             "hard_cap": self.hard_cap,
94             "inactivity_end": self.inactivity_end,
95             "inactivity_rally_thresh": self.inactivity_rally_thresh,
96             "inactivity_min_density": self.inactivity_min_density,
97             "blob_fallback": self.blob_fallback,
98             "blob_factor": self.blob_factor,
99         }
100
101
102     def _served_preset_from_defaults() -> WindowingPreset:
103         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106         - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
107         - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108         - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
109
110         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
111         point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
112         so the two can never silently diverge again.
113         """
114         idx = IndexingConfig() # construct with defaults ? the served operating point
115         return WindowingPreset(
116             name="served",
117             velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet

```

```

default 0.02
118         merge_gap=float(idx.dead_time_merge_gap),
119         min_window_duration=float(idx.min_action_window_duration),
120         max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)
121         hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback
(default OFF)
122         inactivity_end=float(idx.window_inactivity_end), # R4 rally-end trim
(default 0=OFF)
123         inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
124         inactivity_min_density=float(idx.inactivity_min_density),
125         blob_fallback=bool(idx.window_blob_fallback), # R5 blob-fallback
(default OFF)
126         blob_factor=float(idx.window_blob_factor),
127     )
128
129
130     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
131     #: only promotable if it holds up HERE ? the operating point real users get.
132     SERVED: WindowingPreset = _served_preset_from_defaults()
133
134     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
135     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
136     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
137     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
150         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
151         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
152         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
153         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
154         """
155         idx = _as_mapping(config, "indexing")
156         family_cfg = _as_mapping(idx, family)
157         return WindowingPreset(
158             name=f"served:{family}",
159             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
160             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162             max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163             hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164             inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165             inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",

```

```

SERVED.inactivity_rally_thresh)),
166         inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
167         blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168         blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169     )
170
171
172     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
173         """`obj[key]` as a plain mapping, whether `obj` is a dict or a typed config
section
174         (`AppConfig.get` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
175         if obj is None:
176             return {}
177         if isinstance(obj, Mapping):
178             sub = obj.get(key, {})
179         elif hasattr(obj, "get"):
180             sub = obj.get(key, {})
181         else:
182             sub = getattr(obj, key, {})
183         return sub if isinstance(sub, Mapping) else {}
184
185
186     def build_windows(points: List[Any], fps: float, frame_width: float,
187                     preset: WindowingPreset) -> List[Dict[str, Any]]:
188         """Trajectory `points` ? candidate windows under `preset` (the one windowing
call site).
189
190         A thin wrapper over `TrackNetRunner.trajectory_to_action_windows` that takes a
named preset
191         instead of three loose floats, so eval candidate-building and the served path
reference the
192         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195         vt, mg, mwd = preset.as_tuple()
196         return TrackNetRunner.trajectory_to_action_windows(
197             points, fps=fps, frame_width=frame_width,
198             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199             max_window_duration=preset.max_window_duration,
200             window_hard_cap=preset.hard_cap,
201             window_inactivity_end=preset.inactivity_end,
202             inactivity_rally_thresh=preset.inactivity_rally_thresh,
203             inactivity_min_density=preset.inactivity_min_density,
204             window_blob_fallback=preset.blob_fallback,
205             window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """`[{start, end, ...}]` ? `[(start, end)]` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211

```

22. assistant (2026-06-19T17:00:23.385Z)

I have a strong grounding now. The windowing.py docstring gives me a documented precedent that's directly relevant: Gate-A scored +0.074 on the proposal set but -0.008 on the served set ? a real overfitting/transfer failure. Let me check the score floor in the tracker config and the experiment harness for sample sizes (n) to quantify overfitting risk, plus check how many multicourt golden videos exist.

23. user (2026-06-19T17:00:31.299Z)

```
primary-secondary pixel distance: n=9617 med=1 p25=0 p75=2
near(<30px, jitter/same-blob): 9172 mid(30-200): 127 far(>200px, distinct location): 318
=> far+mid frac (genuinely separate candidates): 4.6%
```

24. user (2026-06-19T17:00:32.333Z)

```
1: """Pydantic models for the application configuration.
3: This module defines the canonical, validated, typed configuration for the
6: The models support graceful migration from old config.json files via extra="allow".
18: class ValidationRelevanceConfig(BaseModel):
24: class ValidationConfig(BaseModel):
28:     min_blur_threshold: float = 20.0
30:     relevance: ValidationRelevanceConfig = Field(default_factory=ValidationRelevanceConfig)
33: class TrackNetConfig(BaseModel):
43:     velocity_threshold: float = 0.02
54: class WasbIndexConfig(BaseModel):
55:     """Typed config for the live WASB shuttle substrate (indexing.wasb).
57:     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
59:     because nested config sections defaulted to extra='ignore'.
69:     velocity_threshold: float = 0.02
80:     # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms it.
84: class FusionConfig(BaseModel):
85:     """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
90:     threshold is raised (`min_crossings`/`min_players` default 0 = OFF). The court mask
96:     # Windowing velocity threshold for the fusion family (the engine reads
97:     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98:     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99:     velocity_threshold: float = 0.02
117: class IndexingConfig(BaseModel):
120:     motion_threshold: float = 0.08
126:     detector_score_threshold: float = 0.5
127:     yolo_velocity_threshold: float = 0.15
190:     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
191:     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
192:     fusion: FusionConfig = Field(default_factory=FusionConfig)
202:     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
```

[Showing results with pagination = limit: 30]

25. assistant (2026-06-19T17:00:38.272Z)

A pivotal finding that nuances the noise story in BOTH directions. On GX010142 as a whole, 95% of secondary candidates are within 30px of the primary (jitter/same-blob duplicates of the SAME shuttle, not a second shuttle or distinct noise) ? so over the full clip the "two shuttles at once" framing is rarer than the headline 82% suggests. But the brief's measured window (21-25s) specifically has a spatially-distinct adjacent-court shuttle. The 4.6% "genuinely separate" candidates are exactly where the design lives ? and that's where noise tracks ALSO live. Let me check the experiment harness for the corpus size (n) that governs overfitting risk, and confirm the score floor.

26. user (2026-06-19T17:00:45.790Z)

```
FAR (>200px) secondary candidates: n=2058
  their scores: med=11.98 p25=6.12 p75=13.98 min=0.50 max=17.08
  frac of far-secondaries with score>=10 (confident, indistinguishable from a real shuttle):
62.1%
  frac near floor (<=2): 12.7%
```

27. user (2026-06-19T17:00:46.816Z)

```
10:> [EVALUATION.md](EVALUATION.md); the no-regression gate + golden labels + `baseline.json`
come from
11:> [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md); the modular-cue architecture is
**ADR-007**
```

```

29:2. Test it **offline against golden labels** (most experiments never touch the served path at all).
95:- `label_candidates(candidate_intervals, gt_intervals, iou_threshold=0.5) -> List[int]` (:50)
? y-labels by IoU match to golden (same matcher as the evaluator).
96:- `build_training_set(candidate_rows, gt_intervals, iou_threshold=0.5,
feature_fn=extract_yolo_features) -> (X, y, intervals)` (:64) ? pluggable `feature_fn`.
101:- `backend/eval/calibrate_wasb.py` ? `run(...)` , `load_golden_gts()`;
`backend/eval/gt_loader.py::load_gt_intervals(csv_path, *, strict=True)` is THE canonical golden
reader.
134:+ `calibration.leave_one_video_out()` + `metrics.score()`, graded vs golden
(`gt_loader.load_gt_intervals`).
154:default **only if** it doesn't regress the golden set beyond `tolerance` (exit 1 blocks).
Control is
187:4. **CI promotion lane** using `run_regression.py` against the golden `baseline.json`; write
the
195: `GOLDEN_SET_VENDORING.md`. *(this deliverable; docs-only, no code.)*
203:- **P3 ? CI promotion gate**: `run_regression.py` lane on the golden `baseline.json`;
experiment-leaderboard JSON.
223: `MULTI_SIGNAL_FUSION_PLAN.md` and `GOLDEN_SET_VENDORING.md`.

```

28. assistant (2026-06-19T17:00:52.555Z)

This is decisive for the noise question. Among the spatially-distinct (>200px) candidates ? the exact set the multi-target linker operates on ? 62% have detector scores ≥ 10 , statistically indistinguishable from the primary shuttle's confidence. The detector's own confidence cannot separate a real second shuttle from confident noise (court-line corner, reflection, a player's white shirt/shoe that the WASB heatmap fires on). So the burden falls entirely on the heuristic selection objective. Let me confirm the golden-set sample size (n) which bounds the overfitting risk, and check the LOVO / leave-one-video-out provision.

29. user (2026-06-19T17:00:53.993Z)

```

docs\ANALYZERS_AND_RECONCILERS.md:475:- **The honest measuring stick.** Interval-level, scarce
ground truth + **leave-one-VIDEO-out** (group by
docs\ANALYZERS_AND_RECONCILERS.md:714: RLOOCV interacts badly with grouped folds at N=6 (the
cited study is record-wise, not group-wise).
docs\ABLATION_LAYER.md:46: / `structural-protected`. One wash at n=6 is `inert` (a candidate),
never "irrelevant".
docs\CODE_MAP.md:166:- `@backend/eval/calibration.py` ? overfit guard. `::cross_validate`
(within-video k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out`
(honest cross-video; needs ?2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on
full GT), `::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`),
`::evaluate`. $ `PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ?
`generalization_gap >= 0.1` = overfit alarm.
docs\CODE_MAP.md:176:[Omitted long matching line]
docs\CODE_MAP.md:178:[Omitted long matching line]
docs\CODE_MAP.md:362:| `test_calibrate_local.py` | `eval.calibrate_local` local grid + crossing
gate, leave-one-video-out |
docs\EXPERIMENT_HARNESS.md:69:- `leave_one_video_out(videos, configs, metric="f1",
iou_threshold=0.5) -> dict` (:113) ? **LOVO: pick on other videos, score on held-out video
(honest cross-video).**
docs\EXPERIMENT_HARNESS.md:134:+ `calibration.leave_one_video_out()` + `metrics.score()`, graded
vs golden (`gt_loader.load_gt_intervals`).
docs\GOLDEN_VIDEOS.md:37:| 6 | mahadevpura_singles | badminton (S) | 31 | 29.97 | (original 6) |
? | pre-06-16 |
docs\GOLDEN_VIDEOS.md:38:| 7 | mahadevpura_2 | badminton (multicourt) | 40 | 59.94 | `?/Roora
Request - June 15/[Done] Video 3 ?/mahadevpura-2_proxy.mp4` | `038e0e0915a2` | 2026-06-16 |
docs\GOLDEN_VIDEOS.md:39:| 8 | GX010128 | badminton (multicourt) | 52 | 59.94 | `?/Golden Label
Videos Request - June 15/[Done] Video 13 ?/GX010128.MP4` | `db84f3e65318` | 2026-06-16 |
docs\GOLDEN_VIDEOS.md:40:| 9 | mahadevpura_1 | badminton (S, continuous) | 10 | 59.94 |
`?/[Done] Video 14 - Badminton/mahadevpura-1.MP4` | `38bef75d78cb` | 2026-06-17 |
docs\GOLDEN_VIDEOS.md:52:- **Labeled badminton dirs:** Video 3 (=mahadevpura-2), 6 (=GX030094),
10 (=BXH 2), 13 (=GX010128, a
docs\GOLDEN_VIDEOS.md:53: re-label of #8), 14 (=mahadevpura-1). **Video 13 is a duplicate of
#8** (already ingested).

```


docs\NEXT_STEPS.md:35:sign test 5/6 wins $p=0.109$? honestly not significant, ship=False). The $n=6$ owned corpus is in the collector**

docs\NEXT_STEPS.md:40:| Path | Mahadevpura (clean-ish) | Kushagra (multi-court) |

docs\NEXT_STEPS.md:43:| **Gen-0 owned head** (LOVO, $n=6$) | 0.744 | 0.561 |

docs\NEXT_STEPS.md:49:the exact contrast-metric confound). Gen-0 is 0.10 behind with only $n=6$? a corpus-growth-sized gap, not an

docs\NEXT_STEPS.md:103: `output/MahadevpuraSingles\_highlights.mp4`). **Field kit COMPLETE (2026-06-12, owner ratified the

docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:5:**Grounded in:**

`docs/archives/research/OWNED\_MODEL\_TRAINING\_STUDY.md`, the $n=6$ LOVO + learned prototype

docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:109:### M0.2 ? Grow the corpus . **DONE ($n=6$), running**

. *owner action + my scoring loop*

docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:110:6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-match + LOVO recorded; the

docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:130:reusing `metrics.match\_intervals`. **Define the ship-gate target up front** (open item): a FLOOR of "beat the honest $n=6$

docs\MULTI_SIGNAL_FUSION_PLAN.md:155:- Use the **leave-one-video-out (LOVO)** protocol that

`distill\_local.py` **already implements**:

docs\PMF_FIELD_SIGNALS.md:130: (`output/MahadevpuraSingles\_highlights.mp4`, trimmed to ~60s) saved offline on

docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md:59:`calibrate\_wasb.py` do honest leave-one-video-out (LOVO), but nothing *enforces*

docs\archives\decisions\DECISIONS.md:304>windowing ceilings around F1 ? 0.44 and the learned student *underperforms* the threshold baseline at $n=2$

docs\PROMINENT_COURT_DETECTION.md:16:[Omitted long matching line]

docs\PROMINENT_COURT_DETECTION.md:44:1. **Configured polygon** (MVP, `[design]?buildable now`): a normalized `[x,y,?]` court polygon per camera/clip, exactly the existing

`FusionConfig.court\_polygon`. Draw it once from the dense shuttle/player cluster. Good enough to validate the whole concept on GX010142 + the multicourt golden clips.

[Showing results with pagination = limit: 30]

30. assistant (2026-06-19T17:00:59.688Z)

I now have the full grounding I need. The corpus is $n=6-9$ golden videos, with only ~4 multicourt clips ? and the documented precedent (Gate-A: +0.074 proposal / -0.008 served; the $n=6$ sign-test 5/6 $p=0.109$ "not significant") establishes the exact overfitting risk. The selection objective has free parameters (motion-energy threshold, vertical-excursion threshold, net-crossing deadband, top-K, track-linking gates) that would be tuned on this tiny corpus.

I have enough to deliver a grounded adversarial review on the NOISE problem.

31. user (2026-06-19T17:01:57.903Z)

Structured output provided successfully

32. assistant (2026-06-19T17:02:13.177Z)

I've completed the adversarial review of the NOISE problem, grounded in the actual code and the real GX010142 detector cache.

Key findings (all measured from

`output/wasb/GX010142\_1080p\_c4cfa94a792f\_wasb\_wasbcache/det\_raw.jsonl`):

- **The detector's confidence cannot separate shuttle from noise.** Among spatially-distinct secondary candidates ($>200px$ from the primary ? the exact set the multi-target linker operates on), 62.1% carry score ≥ 10 , indistinguishable from the primary shuttle (primary median 14.4 vs secondary 13.7; median gap only ~1.0 on a 0.5?20 scale). So Layer 1 cannot pre-filter by score, and the entire shuttle-vs-noise burden lands on the heuristic selection objective ? the design's weakest point.

- **A useful nuance the brief understated:** across the full clip, ~95% of secondary candidates are within 30px of the primary (jitter/same-blob duplicates of the *same* shuttle, not a second one). The genuine "two distinct shuttles/blobs" case is ~4.6% of frames ? which is precisely

where both the recovered prominent rally *and* the high-confidence noise live together.

- **The overfitting risk is concrete, not hypothetical.** The objective has ~6?10 free knobs (motion-energy, vertical-excursion, net-crossings deadband/min, floor-contact dwell, court polygon, plus linker top-K/coast/min-duration), tuned against only n=6?9 golden videos (~4 multicourt). The repo already recorded this exact failure: `backend/eval/windowing.py` documents Gate-A scoring +0.074 on PROPOSAL but ?0.008 on SERVED, and `docs/NEXT\_STEPS.md` records the n=6 sign test (5/6, p=0.109) as "not significant."

Verdict: design-needs-revision ? not broken (no `kills\_design`). Motion-energy/vertical-excursion are mimickable by moving noise; the robust pillars are the structural/geometric ones (net-crossings, floor-contact, prominent-court polygon). Required revisions: a candidate-level noise audit, a track-level selection-precision metric (not just end-to-end F1), mandatory LOVO with the generalization-gap alarm, freezing thresholds before tuning on the motivating clip GX010142, and staging the heuristic-to-propose / small-learned-ranker-to-select fork with per-track features persisted now.

One process note: `git pull --ff-only` failed at session start (the remote ref `refs/heads/docs/court-floor-rally-end` wasn't fetched), so I reviewed strictly against local state ? flag if the local checkout may lag remote.